Specialize the First Stage: Fine-Tuning a Cascade's Decision Model on One Game Master's Rulings, Predicted by Replay and Checked Live

Kostadis Roussos
Independent researcher
kostadis@gmail.com

October 2026

Abstract

A cascade's savings are set by its first stage: every item the cheap model cannot settle is paid for twice. Two earlier studies on a tabletop role-playing campaign pipeline evaluated off-the-shelf non-generative decision models against a generative model [1], and designed a Decision-2.0-Nox-4B → `qwen3.8-flash-next` cascade by replaying their frozen outputs, then checked it live [2]. The first study also fine-tuned Nox on the game master's (GM's) rulings but never put the tuned model in the cascade. Here we do, asking whether a domain-specialized first stage removes more of the second. Because the frozen dataset caches Nox's backbone features, tuning its decision head and replaying the resulting cascade takes about a minute of CPU time per variant, cross-fitted so that no item's label reaches its own head or its own gate. On triage, the tuned head lowered escalation from 33% to 25% and raised the cascade's predicted single-stream speed-up from 2.07× to 2.55×, while still missing no more real names than the generative model. On fact typing the replay first showed the second stage almost vanishing, then exposed why: every fact's label is its entity's label and entities recur, so a plain lookup of the GM's earlier rulings scores 99.0%, above every model. Held out by entity, the tuned Nox alone tied the generative model on entities it had never seen (88.4% vs 88.3%). Holding out by token on triage broke the missed-name constraint. We then served the tuned heads on the real two-box system, with gates and predictions pre-registered by hash. Live, the triage cascade scored +2.9 points above qwen (CI [+1.9, +4.2]; predicted +2.8), escalated 24.7% of tokens, missed exactly as many names as qwen, and was 2.51× faster single-stream (predicted 2.38×); the entity-disjoint fact-typing cascade escalated 4.3% and was 4.49× faster (predicted 4.36×) at +1.1 points (CI [-0.1, +2.3]). The replay routed 100.0% and 99.9% of items as the live system did. A specialized first stage removes most of the second on fact typing and a quarter of it on triage, where the second stage stays because the tuned model alone drops twice as many real names. Set beside Vela 2.0, a full fine-tune of the same Nox for safety and routing, the head-only gain on our tasks (+5.5 AUC points on triage) is of the same order as Vela's mid-range gains on its own, at a small fraction of the effort; Vela itself, tuned for another domain, is a worse first stage here than the Nox it came from.

1 Introduction

A reader of the cascade study [2] asked the natural follow-up: if the small model's accuracy gap is what forces escalation, why accept its baseline accuracy? For a bounded classification task one

can trade general-purpose performance for domain specialization by fine-tuning, and a specialized first stage might close enough of the gap to remove the second stage for more of the work. The question has a practical edge in this pipeline. The decision model answers in about 80–150 ms on a DGX Spark and the generative model in about half a second, so each percentage point of escalation removed is latency and queue capacity returned.

Both ingredients already existed. The companion study's E6 [1] fine-tuned Nox's head (and a LoRA) on the GM's rulings and reported that head-only tuning *recalibrates* the readout: triage AUC rose from 0.900 to 0.963. But it scored the tuned model as a standalone classifier. The cascade study's replay [2] showed that a cascade's accuracy, escalation and latency are a pure function of saved per-item outputs when the first stage is deterministic, and that the prediction held live. Here we combine them, with three aims:

1. measure how much a tuned first stage shrinks the second, at the cascade's own objective (the generative model's accuracy, with real names never dropped more often than it drops them);
2. find where the gain comes from, and in particular whether it is specialization or memorization of rulings the GM has already made;
3. check the replay's prediction for a *trained* first stage on the real system, as the cascade study did for an off-the-shelf one.

Findings.

- **Triage.** Tuning removes a quarter of the escalations (33% to 24.7%) at the missed-name constraint, and the cascade ends up +2.9 points *more* accurate than qwen alone, live. The second stage stays: tuned Nox alone is more accurate still (89.6%) but drops 10 real names to qwen's 5.
- **Fact typing.** Most of the apparent gain is memorization: the label is an entity-level ruling, and a lookup of earlier rulings scores 99.0%. Held out by entity, the gain that remains is real but small. Tuned Nox alone ties qwen (88.4% vs 87.8% live), so the cascade escalates only 4.3% and runs 4.49× faster than qwen.
- **The replay held for a trained first stage.** Escalation, accuracy difference and single-stream latency were all predicted before any live request, and all landed within about 5% (latency) or half a point (accuracy).
- **Next to Vela 2.0.** Head-only tuning on one GM's rulings gained about as much on our tasks as Vela's full fine-tune gained on its mid-range tasks (+5.5 AUC points on triage), but on fact typing the gain is accuracy on the common types, not macro-F1. Served as a first stage on our tasks, no Vela member improved on released Nox.
- **One token-disjoint variant failed** the missed-name constraint on the replay and never went live; the replay is where that should be found.

2 Background

The pipeline and its frozen dataset. The companion study [1] evaluated decision models, which read a `state` once and return a probability for every option of a typed question through a small head without generating text, on judgment steps of a pipeline that turns recorded play sessions

into campaign records. Its labels are one GM's past rulings, frozen into a versioned, read-only dataset with every model-facing input pre-rendered. We use two of its tasks, unchanged:

E3 triage (864 tokens, 84 real names) is an unknown capitalised token a campaign name, or filler, a rules term or real-world chatter? As in the cascade study we score it as binary accuracy and separately count *missed names*, because a dropped name is a silent deletion nothing downstream recovers.

E4 fact typing (1,691 facts, 92 entities, 61 chapters) file an extracted fact under one of eight types.

The other two tasks of the cascade study are not candidates. On E4 entity typing Nox alone already matched the generative model, and on E2 spelling Nox trailed it by 56 points, a gap that looks structural rather than one of calibration.

The cascade and its replay. Nox answers first; if its gate confidence (top probability, or $\max(p, 1-p)$ for the binary triage question) is below a threshold, the item goes to `qwen3.8-flash-next` with reasoning off. The *parity gate* is the lowest threshold whose cascade accuracy on a training set is at least qwen's, and on E3 misses no more names than qwen. Thresholds are fitted 2-fold by group (sessions for E3, chapters for E4) and each half is scored with the other half's threshold. The replay composes per-item answers, confidences and single-stream service times; the cascade study found it routed items exactly as the live system did and predicted the cascade-minus-qwen accuracy difference within 0.1 points [2].

3 Method

3.1 Tuning the head on cached features

The released Transformers wrapper refuses `train()`, but the runtime's backbone and head are ordinary PyTorch modules. As in E6, we ran Nox's frozen backbone once over every item through the runtime's own loader and renderer and cached the option and query hidden rows the head reads; the released head on those features reproduces the served probabilities (maximum absolute difference 0.0097 on E3 and 0.0 on E4 in this run). Training is head-only, on CPU: valid-option cross-entropy ($-\log \sum_{k \in \text{valid}} p_k$) at the server's temperature, an L2 pull toward the released head (L2-SP [5], weight 0.1), AdamW at 10^{-4}, batch 32, 20 epochs, and weight 4 on real-name tokens in triage. These settings are E6's and were fixed before this study; E6 itself chose them from four candidates on a cross-validation over the same items (different folds), so they carry a little optimism. Training is seeded and deterministic: re-training reproduced every probability exactly. E4 training also uses the companion study's 839-fact control set, whose 54 entities do not occur in the 1,691 test facts.

3.2 Cross-fitting the head and the gate together

A tuned first stage adds a second place where labels can leak: the head as well as the threshold. We nest both inside the replay's existing 2-fold split. For each half h:

- the *test head* for h is trained on the other half only, and gives the first-stage answers and confidences for h;
- the *gate* for h is the parity gate fitted on the other half, using that half's *inner out-of-fold* predictions (3-fold by group within it), so that, like the test predictions, they come from heads that never saw the item's label.

Task	First stage	Nox alone	Cascade	Δ vs qwen (pt) [95% CI]	Esc. (%)	Missed	Speed-up
E3 triage	released	79.2	84.7	+0.0 [-0.3, +0.3]	33	6 / 6	2.07×
	tuned	89.7	87.5	+2.8 [+1.7, +3.9]	25	6 / 6	2.55×
	tuned, disjoint	89.0	88.5	+3.8 [+2.5, +5.3]	16	9 / 6	3.22×
E4 fact typing	released	83.6	88.2	-0.1 [-0.7, +0.5]	40	–	1.70×
	tuned	89.4	89.7	+1.4 [+0.2, +2.7]	1	–	4.98×
	tuned, disjoint	88.4	88.9	+0.6 [-0.6, +1.8]	4	–	4.34×
E4 fact typing	entity lookup	99.0 (coverage 94%)					

Table 1: Replay of the Nox → qwen cascade with the released and the tuned head, held out as in the text (head and gate cross-fitted). qwen alone scores 84.7% on E3 and 88.3% on E4. "Missed" is real names dropped by the cascade / by qwen alone. Speed-up is against qwen alone, single-stream, from service times measured live in the cascade study. "Disjoint" also removes from training every item sharing an entity (E4) or token (E3) with the test half. The lookup row files each fact under the type the GM gave its entity in the other half, falling back to released Nox when the entity is new.

No item's label therefore influences either its own answer or its own threshold.

3.3 Two hold-out regimes

Holding out by session or chapter is the deployment question (*train on the GM's past sessions, run on the next*), but entities and tokens recur across sessions. We therefore also run a *disjoint* variant that drops from each training set every item whose entity (E4) or token (E3, case-folded) occurs in the test half. It answers a different question: does the gain transfer to things the GM has never ruled on? On E4 it keeps 874 / 901 training items per half (mostly the control set) against 1639 / 1730; on E3, 285 / 238 against 468 / 396.

3.4 Serving a tuned head, and the live check

The runtime verifies its model package against a manifest of per-file hashes and a built-in model identity, and runs a golden-answer check before reporting ready, so a tuned head file cannot simply be dropped into the package. Our server wrapper lets the stock runtime load, verify and golden-check the *released* package, then copies the tuned head's weights into the loaded head module in place and logs the file's hash. Backbone, renderer, scheduler and HTTP API are the stock runtime's. Each half needs its own head, so the live run serves four heads in turn on spark2, re-probing 20 items per head against the replay's probabilities before measuring. The second stage is qwen on spark1 in its deployed configuration (MTP-2 speculative decoding, prefix caching on), as in the cascade study.

We took one variant per task live, chosen on the replay before any live request: tuned on E3 (the only tuned E3 variant that held the missed-name constraint) and disjoint on E4 (the variant whose gain is not memorization). Every gate, every head's hash and the replay's predictions were written to a pre-registration record (SHA-256 `f4dfe8a1...`, 2026-10-11T21:02:49+0000) before the first live request. As in the cascade study, we ran tuned Nox alone, the cascade, and qwen alone on every item in the same session, single-stream ($c = 1$) and with 8 concurrent clients ($c = 8$).

4 Replay results

Triage: a smaller second stage. Tuned on the other half's sessions, Nox alone rose from 79.2% to 89.7% binary accuracy, above qwen's 84.7% (Table 1). That does not make it a replacement: alone it drops 10 real names where qwen drops 6, and the binary score rewards confident "not a name" answers, of which there are many. The parity gate handles exactly this. The tuned cascade escalates 25% of tokens instead of 33%, misses 6 names (qwen 6), scores +2.8 points above qwen (CI [+1.7, +3.9]), and is predicted 2.55× faster than qwen single-stream, against 2.07× for the released head.

This is not name memorization. Only 3 of the 84 real names have a token that also occurs in the other half. What recurs is the table's chatter: filler words, rules terms and real-world references come back every session, and learning how this GM labels them is legitimate domain knowledge.

Triage, token-disjoint: the constraint breaks. Removing every training token that also occurs in the test half cuts the training set to 285 / 238 items per half. The replay then escalates only 16% and scores higher still (+3.8 points), but the cascade misses 9 names against qwen's 6. Its gate was fitted to miss no more names than qwen on the other half and did not transfer. With 84 names in all, three names is the difference, but it is the error this pipeline cannot afford, so this variant fails the objective.

Fact typing: the second stage nearly vanishes, for the wrong reason. Tuned on the other half's chapters, Nox alone reaches 89.4%, the parity gate escalates 1% of facts, and the cascade is predicted 4.98× faster than qwen at +1.4 points above it. The frozen dataset shows why. In E4, 100% of facts carry their entity's majority label: the label is an entity-level ruling copied onto each fact. The same entities recur across chapters, so a model trained on other chapters has seen most test entities ruled on already. A lookup that files each fact under the type the GM gave its entity in the other half covers 94% of facts and scores 99.0%, above qwen, tuned Nox and every model in either earlier study. Much of the tuned head's gain is that lookup learned the hard way, and done worse.

Fact typing, entity-disjoint: specialization that transfers. Trained only on entities absent from the test half (largely the control set), tuned Nox alone scores 88.4%, against 83.6% for the released head and 88.3% for qwen. The gain here cannot come from remembering rulings; it is the readout recalibration E6 described, learned from other entities. The parity gate escalates 4%, and the predicted speed-up is 4.34×. Its gate rests on few out-of-fold items (the disjoint regime leaves almost no repair-set training data inside each half), and one half's threshold is 0, so this point is best read as "tuned Nox alone ties qwen on unseen entities" rather than as a finely tuned cascade.

5 Live results

Accuracy and routing. Both cascades did what the replay said (Tables 2 and 3). On triage the cascade scored 87.3% against qwen's 84.4%, +2.9 points (CI [+1.9, +4.2]) against a predicted +2.8, escalated 24.7% of tokens, and missed 5 real names to qwen's 5. Tuned Nox alone scored 89.6% and missed 10, as on the replay. On fact typing the cascade scored 88.9% against 87.8% (+1.1 points, CI [-0.1, +2.3], predicted +0.6), with Nox alone at 88.4%. The live system escalated exactly the items the frozen gates selected on E3, and all but one of 1,691 on E4. Served and replay probabilities differed by at most 0.0237, slightly above the per-head probe's 0.02 tolerance on a few

Task	First stage	Cascade acc. (%)	qwen acc. (%)	Δ (pt) [95% CI]	Esc. (%)	Missed (casc./qwen)	Speed-up c=1	Throughput gain c=8
E3 triage	released†	84.5	84.4		33	5 / 5	2.2×	2.4×
	tuned	87.3	84.4	+2.9 [+1.9, +4.2]	25	5 / 5	2.51×	2.51×
E4 fact typing	released†	87.8	87.8		40	–	1.8×	2.2×
	tuned	88.9	87.8	+1.1 [-0.1, +2.3]	4	–	4.49×	2.96×

Table 2: Live Nox → qwen cascade on the real system (864 triage tokens, 1691 facts, 0 errors). Tuned rows: this study, both halves pooled, each half served with its own head and gate. †Released rows: the cascade study's live run of the same system with the released head [2]. qwen accuracy and speed-ups are against qwen alone measured in the same session. Throughput gain is items per second with 8 concurrent clients, cascade over qwen alone.

Task	Esc. pred.	Esc. agree	max $\lvert\Delta p\rvert$	Answers agree	qwen self-agree	Δ pred.	Δ meas.	Latency err.
E3 triage	24.7%	100.0%	0.0218	99.3%	99.0%	+2.8	+2.9	+5%
E4 fact typing	4.2%	99.9%	0.0237	99.4%	96.9%	+0.6	+1.1	+3%

Table 3: Pre-registered replay predictions against the live run. "Esc. agree": items the live cascade routed as the frozen gate on replay probabilities did. max $\lvert\Delta p\rvert$: largest difference between the served and the replay's first-stage probability over all items. "Answers agree": live cascade answers equal to the replay's. "qwen self-agree": qwen's live answers equal to its frozen ones, the floor on how well any prediction involving qwen can match. Latency error: composed single-stream mean against measured.

items, and the one E4 item near its threshold flipped. The remaining answer disagreement is qwen's: it reproduced its own frozen answers on 99.0% and 96.9% of items.

Latency and throughput. Single-stream, the triage cascade averaged 213 ms per token against qwen's 534 ms, 2.51× faster (predicted 2.38×, mean +5.5% off); the released head's cascade had managed 2.2×. Fact typing averaged 113 ms against 509 ms, 4.49× (predicted 4.36×, +2.9% off), against 1.8× for the released head. With 8 concurrent clients the cascades sustained 2.51× and 2.96× qwen's throughput. On fact typing that is all the first stage can give: the cascade's throughput (11.65 items/s) equals tuned Nox alone's (11.67), so the remaining bottleneck is the decision model, not qwen.

A contaminated first pass. The pre-registered plan measured everything in one session. In the first pass, another client of the author's began sending long generation requests to spark1's qwen partway through: it overlapped the qwen-alone runs on both tasks and the second E4 half's cascade, inflating qwen's single-stream mean to 881 and 808 ms (against 528 and 502 ms in the cascade study) and the triage latency error to +45%. Accuracy and routing were essentially unchanged. We reran exactly those legs, with the same heads and gates, while sampling spark1's running-request count every 2 s; no leg ever had more requests in flight than its own concurrency, and qwen's means returned to 534 and 509 ms. Tables 2 and 3 use the rerun; the contaminated outputs are kept with the results. The episode is the cascade study's caveat in miniature: the replay composes service times from a quiet box, and a shared second stage breaks that assumption without telling anyone.

Model	Task (held out)	Metric	Before	After	Δ (pt)
Nox-4B, tuned head	E3 triage, by session	AUC	0.900	0.955	+5.5
Nox-4B, tuned head	E3 triage, by session	accuracy (%)	79.2	89.7	+10.5
Nox-4B, tuned head	E4 fact typing, by entity	macro-F1	0.739	0.734	-0.4
Nox-4B, tuned head	E4 fact typing, by entity	accuracy (%)	83.6	88.4	+4.7
Nox-4B, tuned head	E4 fact typing, by chapter	macro-F1	0.739	0.750	+1.1
Vela 2.0 4B‡	Prompt attacks, unseen families	AUC	0.792	0.985	+19.4 [+15.1, +23.7]
Vela 2.0 4B‡	Hate speech, Multilingual HateCheck	AUC	0.646	0.769	+12.3 [+11.7, +12.9]
Vela 2.0 4B‡	Request harm, CultureGuard	AUC	0.863	0.898	+3.5 [+3.2, +3.8]
Vela 2.0 4B‡	Request harm, RTP-LX	AUC	0.761	0.774	+1.3 [+0.8, +1.8]
Vela 2.0 4B‡	Fact-check needed	macro-F1	0.501	1.000	+49.9 [+46.1, +53.5]
Vela 2.0 4B‡	Feedback	macro-F1	0.425	0.920	+49.5 [+47.0, +52.2]
Vela 2.0 4B‡	Modality	macro-F1	0.889	1.000	+11.1 [+9.4, +12.7]
Vela 2.0 4B‡	Domain	macro-F1	0.831	0.820	−1.2 [−2.8, +0.5]
Vela 2.0 4B‡	PII, short texts	F1	0.976	0.987	+1.1 [+0.2, +2.1]
Vela 2.0 4B‡	PII, 8K-token documents	F1	0.908	0.905	−0.3 [−3.6, +3.3]
Vela 2.0 4B‡	Hallucination spans	F1	0.875	0.874	−0.1 [−0.7, +0.6]
Vela 2.0 4B‡	RAGTruth	F1	0.743	0.770	+2.7 [+0.4, +5.1]
Vela 2.0 4B‡	Toxic spans	char-F1	0.451	0.379	−7.2 [−8.0, −6.5]

Table 4: What tuning bought on the tuned-for domain. Tuned Nox: this study, Nox alone, from the cross-fitted held-out probabilities (no item scored by a head that saw its label); "before" is the released Nox. ‡Vela 2.0 4B: as published in its evaluation card [4] (revision `d692394`), not re-measured; "before" is the per-task reference model the card compares against (an earlier specialist, a guard model or a span detector), not Nox, whose scores on these tasks are not published. Δ in points (×100 for AUC and F1); Vela's intervals are the card's paired 95% CIs.

6 The tuning outcome next to Vela 2.0's

Vela 2.0 [4] is a published example of the trade the question proposed: vLLM Semantic Router and KR Labs fine-tuned the same Decision-2.0-Nox-4B for safety, prompt-attack, PII, hallucination and request-routing decisions. Its tuning is a full fine-tune of 4,000 steps on a large public and synthetic mixture (plus two span-head stages); ours is the decision head alone, on a few hundred to a couple of thousand of one GM's rulings, in about a minute of CPU time. Table 4 sets the two outcomes side by side, each on its own domain and in the metrics Vela reports: AUC for yes/no decisions, macro-F1 for multi-class ones.

Same order of gain where the comparison is fair. On triage, tuning raised Nox's AUC from 0.900 to 0.955 (+5.5 points) and its accuracy by 10.5 points. That sits with Vela's middle results: request harm (+1.3 and +3.5 AUC points), hate speech (+12.3), RAGTruth (+2.7). Vela's largest gains, +50 points of macro-F1 on fact-check-needed and feedback routing, are against references that scored near chance (0.50 and 0.43), and its +19.4 on prompt attacks is against a guard model on attack families it had not seen; there was much more room to gain than Nox had on our tasks, where the released model already scored 79–84%. On span tasks Vela is mostly level with its references and loses 7.2 points on toxic spans, so not every target of a broad fine-tune improves.

Accuracy is not the whole outcome. On fact typing, the head tuned on other entities raised accuracy by 4.7 points but left macro-F1 unchanged (0.739 to 0.734): the gain is on the common types, and the rare ones did not improve. Even with the familiar entities in the training data, macro-F1 rose only 1.1 points. For a cascade this is the useful kind of gain, because escalation is

Task	First stage	Alone (%)	Cascade (%)	Δ vs qwen (pt) [95% CI]	Esc. (%)	ms/item	Speed-up
E3 triage	Nox 4B, released	79.2	84.7	+0.0 [-0.3, +0.3]	33	83	2.07×
	Nox 4B, tuned	89.7	87.5	+2.8 [+1.7, +3.9]	25	83	2.55×
	Vela 2.0 0.8B	75.6	84.7	+0.0 [-0.7, +0.6]	55	48	1.59×
	Vela 2.0 4B	76.7	84.6	-0.1 [-0.3, +0.0]	52	221	1.07×
	Vela 2.0 9B	77.8	84.5	-0.2 [-0.7, +0.2]	53	405	0.77×
E4 fact typing	Nox 4B, released	83.6	88.2	-0.1 [-0.7, +0.5]	40	94	1.70×
	Nox 4B, tuned, disjoint	88.4	88.9	+0.6 [-0.6, +1.8]	4	94	4.34×
	Vela 2.0 0.8B	68.2	87.9	-0.4 [-0.7, -0.1]	92	52	0.98×
	Vela 2.0 4B	83.6	87.4	-0.9 [-1.7, -0.1]	26	232	1.38×
	Vela 2.0 9B	82.5	88.1	-0.2 [-0.6, +0.2]	72	424	0.64×

Table 5: Vela 2.0 members as the cascade's first stage on our tasks, replay only (same parity gate, 2-fold by group, and qwen service times as Table 1). "Alone": the first stage's own accuracy. ms/item: the first stage's measured single-stream service time (Vela via its reference server, Nox via its runtime). Tuned Nox rows repeated from Table 1 for comparison. Vela served as released (abstain option on); removing it moved no figure by more than 0.4 points.

decided per item and common items are most of the traffic, but it is narrower than the accuracy figure suggests, and it is the metric Vela's card would have reported.

What the two cost. Vela's card measures its own cost: on the Jev Decision Index of 38 general decision benchmarks it scores 31.91 against Nox's 42.55 [4], a quarter of its base's general-decision score. We did not measure the tuned heads outside our tasks; they are not meant to serve any other question, and a tuned head changes every answer the model gives, so in this pipeline it would be served only for these two tasks. The difference in effort is the larger contrast: the head-only recipe needs cached features and labels the pipeline already records, while Vela's needs a training run over the whole backbone.

Pre-tuned for another domain, Vela is a worse first stage here. We also served the three Vela decoder members, pinned and checked against their own recorded card outputs (maximum difference below 0.001), on the frozen E3 and E4 items and replayed each as the cascade's first stage under the same parity gate (Table 5). None improves on the released Nox it was tuned from. On triage every member must escalate more than half the tokens to keep real names at qwen's level (Nox 33%), and the cascade gains nothing; on fact typing Vela 4B escalates less than Nox but its out-of-fold gate falls below qwen (-0.9 points, CI [-1.7, -0.1]), and the other members escalate most facts. The larger member is not better: Vela 9B's confidence separates its right and wrong triage answers worst of all. Vela's reference server is also slower than Nox's runtime (221 against 83 ms per triage item for the 4B), but replaying Vela's routing at Nox's per-item latency leaves the conclusion unchanged on triage (1.48× against 2.07×), and on fact typing the accuracy shortfall remains whatever the speed. Removing Vela's built-in abstain option, or escalating on its abstain probability, changed nothing material. Its tuning bought gains on its own domain and cost it some of the general decision ability that our tasks draw on; tuning toward our own rulings was the specialization that paid here.

7 Discussion

Specialization does shrink the second stage. On triage, a head tuned on a few hundred of the GM's rulings removes about a quarter of the escalations while holding the constraint that

matters, and on fact typing it closes the gap to the generative model for entities it has never seen. For a bounded classification task that is the trade the question proposed, and it costs a minute of CPU time rather than a GPU job, because the backbone is frozen and its features are cached.

But check what is being specialized to. Fine-tuning on a pipeline's own review records can learn two different things: the GM's *conventions*, which transfer, and the GM's *rulings*, which are a table. E4 shows how much the second can masquerade as the first. A leaderboard-style result ("tuned Nox removes 99% of escalations at 5×") would have been true on the replay and misleading as a claim about the model, and the fix is not a better model but the companion study's conclusion: a ruling should be made once and read back. In this pipeline the right fact-typing design is a lookup of the GM's past rulings first, tuned Nox for unseen entities, and qwen only for the uncertain remainder.

The objective, not accuracy, decides. Tuned Nox alone "beats" qwen on E3 by five points of binary accuracy and still cannot replace it, because it drops more real names. Measuring the first stage by escalation at the parity gate, with the expensive error constrained separately, is what separated the useful variant from the two that looked better.

The replay extends to training. The cascade study's replay chose among fixed first stages. With cached features, the same machinery evaluates trained ones, as long as head and gate are cross-fitted together. Seven variants were scored here in a few minutes of CPU time; only the two that survived went to the hardware, and both did there what the replay said they would: the same items escalated, the same accuracy difference within half a point, and the same speed-up within about 5%. The replay's one blind spot was the one the cascade study named, contention on the second stage, and it showed up as a failed measurement rather than a wrong prediction.

8 Limitations

- **Head only.** The backbone was frozen. E6's LoRA predictions exist but use a different fold structure and were not cross-fitted with the gate; a LoRA first stage would also change Nox's latency and determinism, and was not tested.
- **Settings not tuned here, but not innocent.** Hyperparameters were fixed in advance from E6, which chose them on the same items with different folds.
- **Small label sets.** E3 has 84 real names; the disjoint E3 failure is three names. E4's disjoint gate is fitted on few out-of-fold items.
- **Evaluated on the frozen items.** As in the cascade study, the live run uses the same items as the replay, with every label held out from its own head and gate; it is a test of the prediction, not of new sessions.
- **One GM, one campaign.** The specialization is to one person's conventions, which is the point for this pipeline and a limit for any wider claim.
- **Serving path.** The tuned head is swapped into a stock runtime after the released package verifies; the runtime's own integrity and golden checks therefore describe the released head, not the tuned one. Our per-head probe against the replay stands in for them.

9 Conclusion

Can a first stage tuned on the GM's rulings close the gap and remove the second stage? Partly, and the replay says where before the hardware does. On fact typing, held out by entity, tuned Nox ties the generative model, so the cascade sends it 4.3% of facts and runs 4.49× faster, but the larger gain on the GM's familiar entities is a table, and is better kept as one. On triage the tuned model is more accurate than the generative model and still not a replacement, because it drops more real names; as a first stage it cuts a quarter of the escalations and makes the cascade both faster (2.51×) and more accurate than qwen alone. Specialization buys a cheaper cascade, not a single model, and the measure that tells the two apart is the cascade's own objective, not the first stage's accuracy. A published full fine-tune of the same base for another domain, Vela 2.0, gained about as much on its mid-range targets as our head did on ours, and as a first stage for this pipeline it did worse than the base it came from: what to specialize toward mattered more than how hard.

Data and code availability

Head training (`clef/finetune/cascade_head.py`; the evaluation scripts are in `clef/eval/`), the tuned-head server (`serve_tuned.py`), the cross-fitted replay and gate freezing (`cascade_ft.py`), the live driver (`cascade_ft_live.sh`), the rerun of the contaminated legs (`cascade_ft_rerun.sh`), the Vela runs (`vela_run.sh`, `vela_replay.py`, `serve_vela.py`) and the script that generates every number and table in this paper (`ft_paper.py`) are in the public repository [6]. The campaign data is not released because it quotes play sessions verbatim and names real people.

Use of AI assistance

The experiments, analysis and a first draft of this paper were carried out by an AI coding agent (Claude, Anthropic) working under the author's direction, with the author reviewing results and making the decisions recorded in the paper. The author takes full responsibility for the content.

References

[1] K. Roussos. Decision models at the game table: where non-generative classifiers help and fail in LLM tooling for tabletop role-playing campaigns. Manuscript, October 2026. In [6], `paper/main.tex`.

[2] K. Roussos. Simulate before you serve: predicting a decision-model cascade from frozen outputs, and checking the prediction on real hardware. Manuscript, October 2026. In [6], `paper-cascade/main.tex`.

[3] vLLM Semantic Router Team. System One Auto. Blog post, `https://vllm-sr.ai/blog/system-one-auto/`, accessed October 2026.

[4] vLLM Semantic Router Team and KR Labs. Vela 2.0 (0.3B, 0.8B, 4B, 9B): model cards, evaluation and training notes. `https://huggingface.co/vllm-sr/Vela-2.0-4B`, revision `d692394`, accessed October 2026.

[5] X. Li, Y. Grandvalet, and F. Davoine. Explicit inductive bias for transfer learning with convolutional networks. In *ICML*, 2018.

[6] K. Roussos. dgx-fun: experiments with local LLM serving on NVIDIA DGX Spark. https://github.com/kostadis/dgx-fun, 2026.